

Jetpack Compose

Modern Android UI in Kotlin — a colorful pocket guide

🪄 Modifiers
📜 Lists & Grids
🎛️ State
🌀 Side Effects
🎨 Theming
✨ Animations
👆 Gestures
🖌️ Canvas
🧪 Testing

Book 3 · 14 chapters · code + phone-screen previews · exercises & cheat sheet

Contents

01 💭 Thinking in Compose 3

02 🪄 Modifiers in Depth 7

03 🔤 Text & Typography 11

04 📐 Layouts 15

05 📜 Lists, Grids & Pagers 19

06 🎛️ State Deep Dive 24

07 🌀 Side Effects 28

08 🧱 Material 3 Components 32

09 🎨 Theming 37

10 ✨ Animations 41

11 👆 Gestures 47

12 🖌️ Canvas & Custom Layouts 51

13 🧪 Performance & Testing 56

14 🚀 Mini Project: Habit Tracker 60

★ ⚡ Cheat Sheet 65

✓ ✅ Exercise Solutions 66

CHAPTER 1

Thinking in Compose

Declarative UI, composition and recomposition



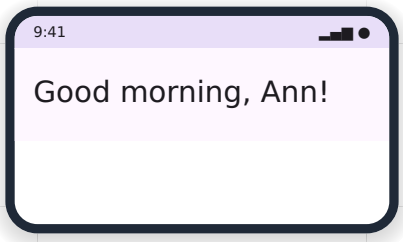
Jetpack Compose is Android's modern UI toolkit. Instead of creating views and changing them step by step (*imperative*), you **describe what the UI should look like for the current data** (*declarative*). When data changes, Compose updates the screen for you.

Old View system	Compose
XML layouts + findViewById	Kotlin functions
textView.text = "Hi"	Text("Hi")
You update views manually	UI = f(state), updates automatically
RecyclerView + Adapter	LazyColumn { items(list) { } }

UI = f(state)

Greeting.kt

```
@Composable
fun Greeting(name: String, isMorning: Boolean) {
    val word = if (isMorning) "Good morning" else
"Hello"
    Text("$word, $name!")
}
```



📱 What you see

The function just turns inputs into UI. No `return`, no view objects — it **emits** UI.

Composition & recomposition

- **Composition** — the first time Compose runs your functions and builds the UI tree.
- **Recomposition** — re-running functions whose inputs or state changed.
- Compose **skips** functions whose inputs didn't change. That's why it's fast.

Recompose.kt

```kotlin
@Composable
fun Screen() {
    var clicks by remember { mutableStateOf(0) }
    Column {
        Header()                        // skipped: no
changing input
        Text("Clicks: $clicks")         // recomposes
on click
        Button(onClick = { clicks++ }) { Text("Tap") }
    }
}

@Composable
fun Header() {
    Log.d("Compose", "Header composed")
    Text("My App", fontSize = 24.sp)
}
```

▶ Logcat

```
D/Compose: Header composed
   (tap, tap, tap... nothing more logged)
```

The rules of composables

📌 **Remember**

1. Composables can run **often** and in **any order**.
2. They should be **fast** — no network or disk work inside.
3. They must be **side-effect free** — don't change variables outside or call APIs directly. Use effects (Chapter 7) for that.

⚠️ **Watch out**

Never do this — it runs on every recomposition:

Bad.kt

```
@Composable
fun Bad(vm: MyViewModel) {
    vm.loadData()          // ❌ called again and
again
    var count = 0
    Button(onClick = { count++ }) {  // ❌ not state:
UI never updates
        Text("$count")
    }
}
```

Naming conventions

Kind	Style	Example
UI composables	PascalCase noun	`UserCard()`
Return a value	camelCase	`rememberScrollState()`
First optional param	`modifier`	`Modifier = Modifier`

CHAPTER 2

Modifiers in Depth

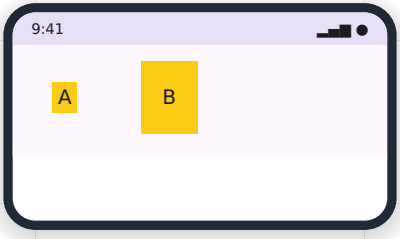
Order, chaining, shapes, gradients and custom modifiers

Modifiers are applied **in order, from outside to inside**. Changing the order changes the result!

Order.kt

```
// A: padding outside the background
Text("A", Modifier
    .padding(16.dp)
    .background(Color.Yellow))

// B: padding inside the background
Text("B", Modifier
    .background(Color.Yellow)
    .padding(16.dp))
```



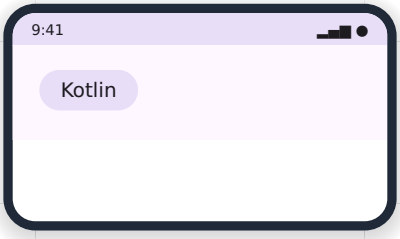
📱 What you see

Always accept a modifier

Chip.kt

```
@Composable
fun Chip(text: String, modifier: Modifier = Modifier)
{
    Text(
        text,
        modifier = modifier                  //
caller's first
            .clip(RoundedCornerShape(50))
            .background(Color(0xFFE8DEF8))
            .padding(horizontal = 12.dp, vertical =
6.dp)
    )
}

// caller decides the outer layout:
Chip("Kotlin", Modifier.padding(4.dp))
```

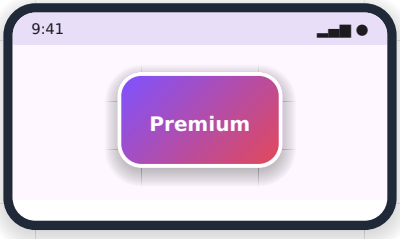


📱 What you see

Shapes, borders, shadows & gradients

Fancy.kt

```
Box(
    Modifier
        .size(120.dp, 70.dp)
        .shadow(8.dp, RoundedCornerShape(16.dp))
        .background(
            Brush.linearGradient(
                listOf(Color(0xFF7F52FF),
Color(0xFFE44857))
            ),
            RoundedCornerShape(16.dp)
        )
        .border(2.dp, Color.White,
RoundedCornerShape(16.dp)),
    contentAlignment = Alignment.Center
) {
    Text("Premium", color = Color.White,
         fontWeight = FontWeight.Bold)
}
```



📱 What you see

Shape	Looks like
RoundedCornerShape(12.dp)	Rounded corners
CircleShape	Perfect circle
CutCornerShape(8.dp)	Cut corners
RoundedCornerShape(50)	Pill (50%)

Transformations with graphicsLayer

Layer.kt

```
Image(
    painter = painterResource(R.drawable.card),
    contentDescription = null,
    modifier = Modifier.graphicsLayer {
        rotationZ = 8f
        scaleX = 1.1f
        scaleY = 1.1f
        alpha = 0.9f
    }
)
```

Custom & conditional modifiers

CustomModifiers.kt

```
// Reusable modifier as an extension function
fun Modifier.card() = this
    .shadow(4.dp, RoundedCornerShape(12.dp))
    .background(Color.White,
RoundedCornerShape(12.dp))
    .padding(16.dp)

// Apply only sometimes
fun Modifier.thenIf(
    condition: Boolean,
    other: Modifier.() -> Modifier
) = if (condition) this.other() else this

Text("Hi", Modifier
    .card()
    .thenIf(isSelected) { border(2.dp, Color.Blue) })
```

> 💡 **Tip**
>
> Clicks with no ripple feedback feel broken. `clickable` adds ripple automatically; put it **after** `clip()` so the ripple follows the shape.

CHAPTER 3

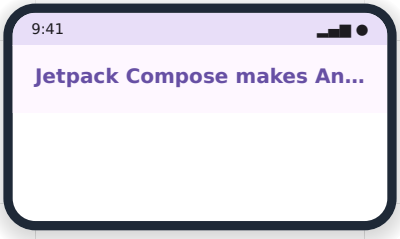
Text & Typography

Styling, rich text, fonts and overflow

Text parameters

TextParams.kt

```
Text(
    text = "Jetpack Compose makes Android UI easy and
fun",
    style = MaterialTheme.typography.titleMedium,
    color = MaterialTheme.colorScheme.primary,
    maxLines = 1,
    overflow = TextOverflow.Ellipsis,
    textAlign = TextAlign.Center,
    letterSpacing = 0.5.sp
)
```

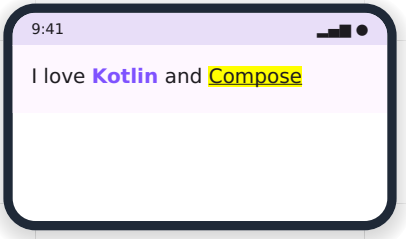


📱 What you see

Rich text with AnnotatedString

RichText.kt

```
val text = buildAnnotatedString {
    append("I love ")
    withStyle(SpanStyle(
        color = Color(0xFF7F52FF),
        fontWeight = FontWeight.Bold)) {
        append("Kotlin")
    }
    append(" and ")
    withStyle(SpanStyle(
        background = Color.Yellow,
        textDecoration = TextDecoration.Underline)) {
        append("Compose")
    }
}
Text(text)
```



📱 What you see

Material type scale

Style	Size	Use for
displayLarge	57sp	Huge numbers
headlineMedium	28sp	Screen titles
titleLarge	22sp	App bar titles
titleMedium	16sp	Card titles
bodyLarge	16sp	Main text
bodyMedium	14sp	Secondary text
labelLarge	14sp	Buttons
labelSmall	11sp	Captions

Custom fonts

Type.kt

```
// put files in res/font/
val Poppins = FontFamily(
    Font(R.font.poppins_regular, FontWeight.Normal),
    Font(R.font.poppins_bold, FontWeight.Bold)
)

val AppTypography = Typography(
    headlineMedium = TextStyle(
        fontFamily = Poppins,
        fontWeight = FontWeight.Bold,
        fontSize = 28.sp
    ),
    bodyLarge = TextStyle(fontFamily = Poppins,
fontSize = 16.sp)
)

MaterialTheme(typography = AppTypography) { /* app
*/ }
```

Selectable text & strings

Strings.kt

```
SelectionContainer {
    Text("Long-press to copy this text")
}

// Use string resources for translation:
Text(stringResource(R.string.welcome, userName))
// strings.xml: <string name="welcome">Hello, %1$s!</
string>
```

> 💡 **Tip**
> Always prefer `MaterialTheme.typography` styles over hard-coded font sizes — your whole app stays consistent and supports the user's font-size settings.

CHAPTER 4

Layouts

Weight, FlowRow, adaptive layouts and scaffolds

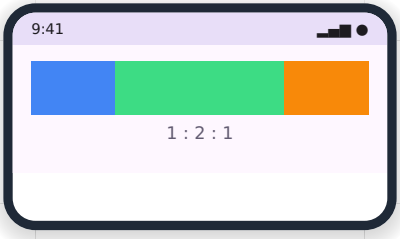
Quick recap

Layout	Direction	Main options
Column	↓ vertical	verticalArrangement, horizontalAlignment
Row	→ horizontal	horizontalArrangement, verticalAlignment
Box	⧉ stacked	contentAlignment, Modifier.align()

weight — share space

Weight.kt

```
Row(Modifier.fillMaxWidth().height(40.dp)) {
    Box(Modifier.weight(1f).fillMaxHeight()
        .background(Color(0xFF4285F4)))
    Box(Modifier.weight(2f).fillMaxHeight()
        .background(Color(0xFF3DDC84)))
    Box(Modifier.weight(1f).fillMaxHeight()
        .background(Color(0xFFF88909)))
}
```



📱 What you see

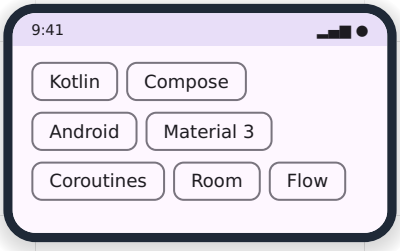
Arrangements

Arrangement	Effect
Start / End / Center	Group together
SpaceBetween	Edges + equal gaps
SpaceEvenly	Equal gaps everywhere
SpaceAround	Half gaps at edges
spacedBy(8.dp)	Fixed gap between items

FlowRow — wrap to next line

Tags.kt

```
@OptIn(ExperimentalLayoutApi::class)
@Composable
fun Tags(tags: List<String>) {
    FlowRow(
        horizontalArrangement =
Arrangement.spacedBy(6.dp),
        verticalArrangement =
Arrangement.spacedBy(6.dp)
    ) {
        tags.forEach { AssistChip(onClick = {}, label
= { Text(it) }) }
    }
}
```



📱 What you see

Adaptive with BoxWithConstraints

Adaptive.kt

```kotlin
@Composable
fun AdaptiveScreen() {
    BoxWithConstraints {
        if (maxWidth < 600.dp) {
            // phone: list only
            ItemList()
        } else {
            // tablet / foldable: list + details
            Row {
                ItemList(Modifier.weight(1f))
                ItemDetails(Modifier.weight(2f))
            }
        }
    }
}
```

Scrolling a normal Column

Scroll.kt

```kotlin
Column(
    Modifier
        .fillMaxSize()
        .verticalScroll(rememberScrollState())
        .padding(16.dp)
) {
    repeat(30) { Text("Line $it") }
}
```

> ⚠️ **Watch out**
> For long or dynamic lists, use `LazyColumn` instead — a scrolling Column creates **all** children at once.

Window insets

Insets.kt

```
// with enableEdgeToEdge(), keep content out of system
bars:
Column(Modifier
    .fillMaxSize()
    .windowInsetsPadding(WindowInsets.systemBars)
    .imePadding())          // move up when keyboard
opens
```

> 💡 **Tip**
> Inside a `Scaffold`, the `innerPadding` already includes the system bars — just apply it.

CHAPTER 5

Lists, Grids & Pagers

LazyColumn, sticky headers, grids and swiping pages



LazyColumn with keys

Contacts.kt

```
data class Contact(val id: Int, val name: String)

@Composable
fun Contacts(list: List<Contact>, onClick: (Contact) -
> Unit) {
    LazyColumn(
        contentPadding = PaddingValues(16.dp),
        verticalArrangement =
Arrangement.spacedBy(8.dp)
    ) {
        items(list, key = { it.id }) { c ->
            ListItem(
                headlineContent = { Text(c.name) },
                leadingContent = { Avatar(c.name) },
                modifier = Modifier
                    .clickable { onClick(c) }
                    .animateItem()   // smooth add/
remove/move
            )
        }
    }
}
```

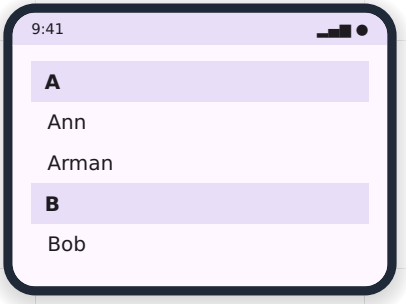
📌 Remember

A stable `key` lets Compose keep each row's state and animate changes correctly when items move.

Sticky headers

Sticky.kt

```kotlin
@OptIn(ExperimentalFoundationApi::class)
@Composable
fun Grouped(contacts: List<Contact>) {
    val groups = contacts.groupBy { it.name.first() }
    LazyColumn {
        groups.forEach { (letter, people) ->
            stickyHeader {
                Text(letter.toString(),
                    Modifier.fillMaxWidth()
                        .background(Color(0xFFE8DEF8))
                        .padding(8.dp),
                    fontWeight = FontWeight.Bold)
            }
            items(people, key = { it.id }) {
                Text(it.name, Modifier.padding(12.dp))
            }
        }
    }
}
```



📱 What you see

Grids

Gallery.kt

```
LazyVerticalGrid(
    columns = GridCells.Adaptive(minSize = 100.dp),
    contentPadding = PaddingValues(8.dp),
    horizontalArrangement =
Arrangement.spacedBy(8.dp),
    verticalArrangement = Arrangement.spacedBy(8.dp)
) {
    items(photos) { photo ->
        AsyncImage(photo.url, null,

Modifier.aspectRatio(1f).clip(RoundedCornerShape(8.dp)),
            contentScale = ContentScale.Crop)
    }
}
```



📱 What you see

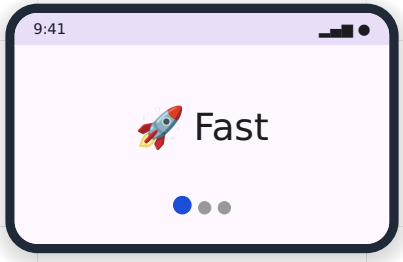
> 💡 **Tip**
> `GridCells.Fixed(3)` always gives 3 columns; `Adaptive` fits as many as possible. For Pinterest-style layouts use `LazyVerticalStaggeredGrid`.

HorizontalPager — swipe pages

Onboarding.kt

```
@Composable
fun Onboarding() {
    val pages = listOf("🚀 Fast", "🎨 Beautiful", "🔒
Safe")
    val pagerState = rememberPagerState { pages.size }

    Column(horizontalAlignment =
Alignment.CenterHorizontally) {
        HorizontalPager(pagerState,
Modifier.weight(1f)) { page ->
            Box(Modifier.fillMaxSize(),
Alignment.Center) {
                Text(pages[page], fontSize = 32.sp)
            }
        }
        Row {     // page indicator dots
            repeat(pages.size) { i ->
                val active = pagerState.currentPage
== i
                Box(Modifier.padding(4.dp)
                    .size(if (active) 12.dp else 8.dp)
                    .clip(CircleShape)
                    .background(if (active)
Color.Blue else Color.Gray))
            }
        }
    }
}
```



📱 What you see

Controlling scroll

ScrollControl.kt

```kotlin
val listState = rememberLazyListState()
val scope = rememberCoroutineScope()

LazyColumn(state = listState) { /* ... */ }

Button(onClick = {
    scope.launch { listState.animateScrollToItem(0) }
}) { Text("↑ Top") }
```

CHAPTER 6

State Deep Dive

remember, derivedStateOf and state holders



API	Purpose
remember { }	Keep a value across recompositions
rememberSaveable { }	Also survive rotation & process death
mutableStateOf(x)	Observable single value
mutableStateListOf()	Observable list
mutableIntStateOf(0)	Observable Int (no boxing)
derivedStateOf { }	Value computed from other state
remember(key) { }	Recalculate when key changes

remember with keys

RememberKey.kt

```
@Composable
fun SortedList(items: List<String>, ascending:
Boolean) {
    // recomputed only when items or ascending change
    val sorted = remember(items, ascending) {
        if (ascending) items.sorted() else
items.sortedDescending()
    }
    sorted.forEach { Text(it) }
}
```

derivedStateOf — avoid extra work

DerivedState.kt

```
@Composable
fun ListWithFab(items: List<String>) {
    val listState = rememberLazyListState()

    // changes only when crossing index 0, not on
every pixel scroll
    val showFab by remember {
        derivedStateOf {
listState.firstVisibleItemIndex > 0 }
    }

    Box {
        LazyColumn(state = listState) {
            items(items) { Text(it,
Modifier.padding(16.dp)) }
        }
        if (showFab) {
            FloatingActionButton(onClick = {},

Modifier.align(Alignment.BottomEnd).padding(16.dp)) {
                Icon(Icons.Default.KeyboardArrowUp,
"Top")
            }
        }
    }
}
```

Saving custom types

Saveable.kt

```
@Parcelize
data class Filter(val query: String, val onlyDone:
Boolean)
    : Parcelable

var filter by rememberSaveable {
    mutableStateOf(Filter("", false))
}
```

Plain state holder classes

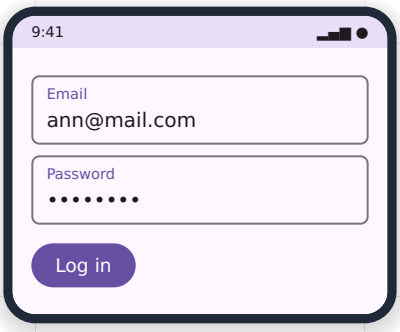
When UI logic gets complex, move it into a small class and `remember` it.

LoginForm.kt

```
class LoginFormState {
    var email by mutableStateOf("")
    var password by mutableStateOf("")
    val isValid: Boolean
        get() = "@" in email && password.length >= 6
}

@Composable
fun rememberLoginFormState() = remember {
LoginFormState() }

@Composable
fun LoginForm(onLogin: (String, String) -> Unit) {
    val form = rememberLoginFormState()
    Column(Modifier.padding(16.dp)) {
        OutlinedTextField(form.email, { form.email =
it },
            label = { Text("Email") })
        OutlinedTextField(form.password, {
form.password = it },
            label = { Text("Password") },
            visualTransformation =
PasswordVisualTransformation())
        Button(onClick = { onLogin(form.email,
form.password) },
               enabled = form.isValid) { Text("Log
in") }
    }
}
```



📱 What you see

📌 Remember

Where should state live?

• Simple UI state (expanded, text) → `remember` in the composable.

• Complex UI logic → a state holder class.

• Business data (from network/DB) → `ViewModel`.

CHAPTER 7

Side Effects

LaunchedEffect, DisposableEffect and friends



A **side effect** is anything that escapes the composable: starting a coroutine, registering a listener, logging analytics. Compose gives safe APIs for each case.

Effect	Use when
LaunchedEffect(key)	Run a suspend block when key changes
rememberCoroutineScope()	Launch coroutines from callbacks (clicks)
DisposableEffect(key)	Register + clean up (listeners)
rememberUpdatedState(x)	Use latest value inside long effect
produceState(init)	Turn async work into State
snapshotFlow { }	Turn Compose state into a Flow
SideEffect { }	Run after every successful recomposition

LaunchedEffect

Launched.kt

```
@Composable
fun SearchResults(query: String, vm: SearchViewModel)
{
    // cancelled & restarted every time query changes
    LaunchedEffect(query) {
        delay(300)            // debounce typing
        vm.search(query)
    }
}
```

DisposableEffect — clean up

Disposable.kt

```
@Composable
fun LifecycleLogger() {
    val owner = LocalLifecycleOwner.current
    DisposableEffect(owner) {
        val observer = LifecycleEventObserver { _,
event ->
            Log.d("Life", "Event: $event")
        }
        owner.lifecycle.addObserver(observer)
        onDispose {
            owner.lifecycle.removeObserver(observer)
        }
    }
}
```

▶ Logcat

```
D/Life: Event: ON_CREATE
D/Life: Event: ON_START
D/Life: Event: ON_RESUME
```

rememberUpdatedState

Splash.kt

```
@Composable
fun SplashScreen(onTimeout: () -> Unit) {
    val currentOnTimeout by
rememberUpdatedState(onTimeout)

    LaunchedEffect(Unit) {       // runs only once
        delay(2000)
        currentOnTimeout()       // but calls the
LATEST lambda
    }
    Text("🚀 Loading...")
}
```

produceState

Produce.kt

```
@Composable
fun UserName(id: Int, repo: UserRepository):
State<String> =
    produceState(initialValue = "Loading...", id) {
        value = try { repo.getUser(id).name }
                catch (e: Exception) { "Error" }
    }

// usage
val name by UserName(42, repo)
Text(name)
```

snapshotFlow

SnapshotFlow.kt

```
val listState = rememberLazyListState()

LaunchedEffect(listState) {
    snapshotFlow { listState.firstVisibleItemIndex }
        .distinctUntilChanged()
        .collect { index -> analytics.log("scrolled
to $index") }
}
```

> ⚠️ **Watch out**
> Don't use `LaunchedEffect(true)` with values that change — pick the right **key**, or the effect will use stale data.

CHAPTER 8

Material 3 Components

Dialogs, sheets, snackbars, tabs and more



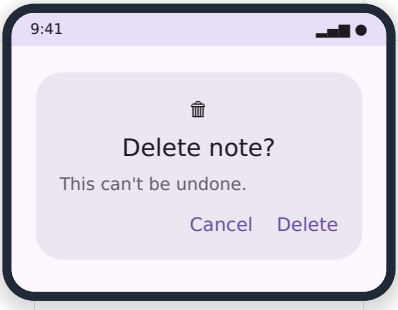
AlertDialog

Dialog.kt

```
var show by remember { mutableStateOf(false) }

Button(onClick = { show = true }) { Text("Delete") }

if (show) {
    AlertDialog(
        onDismissRequest = { show = false },
        icon = { Icon(Icons.Default.Delete, null) },
        title = { Text("Delete note?") },
        text = { Text("This can't be undone.") },
        confirmButton = {
            TextButton(onClick = { show = false }) {
Text("Delete") }
        },
        dismissButton = {
            TextButton(onClick = { show = false }) {
Text("Cancel") }
        }
    )
}
```

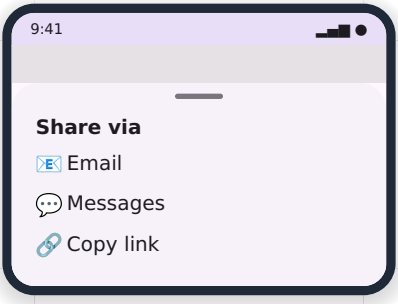


📱 What you see

Modal bottom sheet

Sheet.kt

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ShareSheet(onDismiss: () -> Unit) {
    ModalBottomSheet(
        onDismissRequest = onDismiss,
        sheetState = rememberModalBottomSheetState()
    ) {
        Column(Modifier.padding(16.dp)) {
            Text("Share via", style =
MaterialTheme.typography.titleMedium)
            listOf("📧 Email", "💬 Messages",
"🔗 Copy link")
                .forEach {
                    Text(it, Modifier.fillMaxWidth()
                        .clickable {
onDismiss() }.padding(12.dp))
                }
        }
    }
}
```



📱 What you see

Snackbar

Snackbar.kt

```
val snackbarHost = remember { SnackbarHostState() }
val scope = rememberCoroutineScope()

Scaffold(snackbarHost = {
SnackbarHost(snackbarHost) }) { p ->
    Button(
        onClick = {
            scope.launch {
                val result =
snackbarHost.showSnackbar(
                    "Message archived", actionLabel =
"Undo")
                if (result ==
SnackbarResult.ActionPerformed) {
                    /* undo */
                }
            }
        },
        modifier = Modifier.padding(p)
    ) { Text("Archive") }
}
```

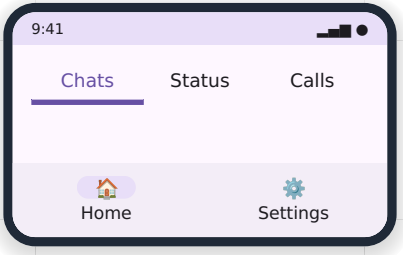
Tabs & bottom navigation

Tabs.kt

```
var tab by remember { mutableIntStateOf(0) }
val titles = listOf("Chats", "Status", "Calls")

PrimaryTabRow(selectedTabIndex = tab) {
    titles.forEachIndexed { i, t ->
        Tab(selected = tab == i, onClick = { tab =
i },
            text = { Text(t) })
    }
}

NavigationBar {
    NavigationBarItem(
        selected = true, onClick = { },
        icon = { Icon(Icons.Default.Home, null) },
        label = { Text("Home") })
    NavigationBarItem(
        selected = false, onClick = { },
        icon = { Icon(Icons.Default.Settings, null) },
        label = { Text("Settings") })
}
```



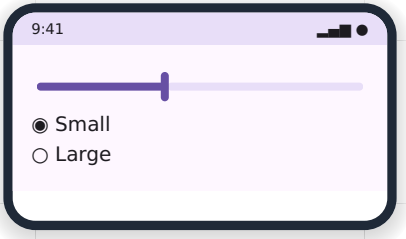
📱 What you see

Inputs

Inputs.kt

```
var volume by remember { mutableFloatStateOf(40f) }
Slider(value = volume, onValueChange = { volume =
it },
       valueRange = 0f..100f)

var selected by remember { mutableStateOf("Small") }
listOf("Small", "Large").forEach { size ->
    Row(verticalAlignment =
Alignment.CenterVertically) {
        RadioButton(selected == size, onClick = {
selected = size })
        Text(size)
    }
}
```



📱 What you see

CHAPTER 9

Theming

Colors, dark mode, shapes and CompositionLocal



A theme gives your whole app consistent **colors**, **typography** and **shapes**. Change it once, and every screen updates.

Color schemes

Theme.kt

```
private val LightColors = lightColorScheme(
    primary = Color(0xFF4285F4),
    onPrimary = Color.White,
    secondary = Color(0xFF3DDC84),
    background = Color(0xFFFDFCFF),
    surface = Color(0xFFFDFCFF),
)

private val DarkColors = darkColorScheme(
    primary = Color(0xFFA8C7FA),
    secondary = Color(0xFF7FE0A8),
)

@Composable
fun AppTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    dynamicColor: Boolean = true,
    content: @Composable () -> Unit
) {
    val colors = when {
        dynamicColor && Build.VERSION.SDK_INT >= 31 -
> {
            val ctx = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(ctx)
            else dynamicLightColorScheme(ctx)
        }
        darkTheme -> DarkColors
        else -> LightColors
    }
```

```
    MaterialTheme(
        colorScheme = colors,
        typography = AppTypography,
        shapes = Shapes(medium =
RoundedCornerShape(16.dp)),
        content = content
    )
}
```

Color roles

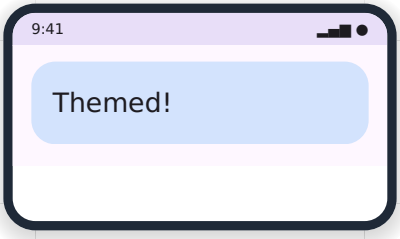
Role	Used for
primary	Main buttons, active states
onPrimary	Text/icons on primary
primaryContainer	Tonal buttons, chips
secondary	Less prominent accents
surface	Cards, sheets, menus
error	Errors & destructive actions

> 📌 **Remember**
> Each color has an **on** pair (e.g. `onSurface`) for readable content on top of it. `Surface` and `Card` pick the right content color automatically.

Using theme values

UseTheme.kt

```
Card(
    colors = CardDefaults.cardColors(
        containerColor =
MaterialTheme.colorScheme.primaryContainer
    ),
    shape = MaterialTheme.shapes.medium
) {
    Text("Themed!",
        Modifier.padding(16.dp),
        style = MaterialTheme.typography.titleLarge)
}
```



📱 What you see

Your own design tokens: CompositionLocal

Spacing.kt

```
data class Spacing(
    val small: Dp = 4.dp,
    val medium: Dp = 8.dp,
    val large: Dp = 16.dp
)

val LocalSpacing = staticCompositionLocalOf {
Spacing() }

@Composable
fun AppTheme(content: @Composable () -> Unit) {
    CompositionLocalProvider(LocalSpacing provides
Spacing()) {
        MaterialTheme(content = content)
    }
}

// anywhere inside:
val spacing = LocalSpacing.current
Text("Hi", Modifier.padding(spacing.large))
```

> 💡 **Tip**
> Preview both themes at once: add `@Preview(uiMode = Configuration.UI_MODE_NIGHT_YES)` next to a normal `@Preview`.

CHAPTER 10

Animations

Make your UI move and feel alive



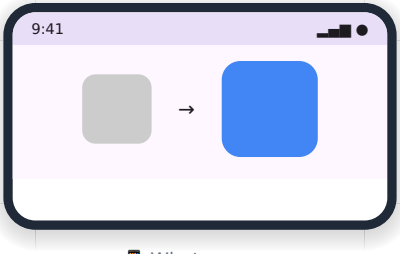
API	Use for
animate*AsState	Animate one value (color, size, float)
AnimatedVisibility	Show / hide with enter & exit
AnimatedContent	Swap content with a transition
Crossfade	Simple fade between screens
animateContentSize()	Smooth size changes
updateTransition	Several values together
rememberInfiniteTransition	Looping animations

animate*AsState

AnimateState.kt

```
@Composable
fun SelectableCard() {
    var selected by remember { mutableStateOf(false) }
    val bg by animateColorAsState(
        if (selected) Color(0xFF4285F4) else
Color.LightGray,
        label = "bg")
    val size by animateDpAsState(
        if (selected) 140.dp else 100.dp,
        animationSpec = spring(dampingRatio = 0.4f),
        label = "size")


Box(Modifier.size(size).clip(RoundedCornerShape(16.dp))
        .background(bg).clickable { selected = !
selected })
}
```



📱 What you see

Animation specs

Spec	Feels like
tween(500)	Fixed duration, easing curve
spring()	Physics, bouncy, natural
keyframes { }	Exact values at exact times
infiniteRepeatable()	Loop forever

AnimatedVisibility

Visibility.kt

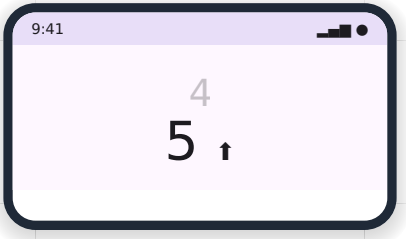
```
var expanded by remember { mutableStateOf(false) }

Column(Modifier.animateContentSize()) {
    TextButton(onClick = { expanded = !expanded }) {
        Text(if (expanded) "Show less ▲" else "Show
more ▼")
    }
    AnimatedVisibility(
        visible = expanded,
        enter = fadeIn() + expandVertically(),
        exit = fadeOut() + shrinkVertically()
    ) {
        Text("Here are the hidden details! 🎉")
    }
}
```

AnimatedContent — animated counter

AnimatedCounter.kt

```
@Composable
fun AnimatedCounter(count: Int) {
    AnimatedContent(
        targetState = count,
        transitionSpec = {
            if (targetState > initialState) {
                slideInVertically { it } + fadeIn()
togetherWith
                    slideOutVertically { -it } +
fadeOut()
            } else {
                slideInVertically { -it } + fadeIn()
togetherWith
                    slideOutVertically { it } +
fadeOut()
            }
        },
        label = "counter"
    ) { value ->
        Text("$value", fontSize = 48.sp)
    }
}
```



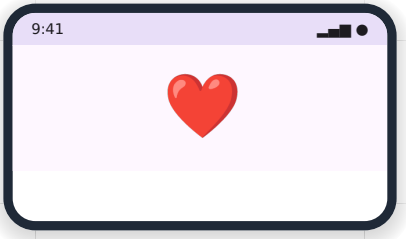
📱 What you see

Infinite animation — pulsing heart

Pulse.kt

```
@Composable
fun PulsingHeart() {
    val t = rememberInfiniteTransition(label =
"pulse")
    val scale by t.animateFloat(
        initialValue = 1f, targetValue = 1.3f,
        animationSpec = infiniteRepeatable(
            tween(600), RepeatMode.Reverse),
        label = "scale")

    Text("❤️", fontSize = 48.sp,
         modifier = Modifier.graphicsLayer {
             scaleX = scale; scaleY = scale
         })
}
```



📱 What you see

> 💡 **Tip**
> Read animated values inside `graphicsLayer { }` or `offset { }` lambdas: Compose then skips recomposition and only redraws — much smoother.

updateTransition — many values at once

Transition.kt

```
enum class FabState { Collapsed, Expanded }

val transition = updateTransition(fabState, label =
"fab")
val rotation by transition.animateFloat(label =
"rot") {
    if (it == FabState.Expanded) 45f else 0f
}
val width by transition.animateDp(label = "w") {
    if (it == FabState.Expanded) 160.dp else 56.dp
}
```

CHAPTER 11

Gestures

Taps, drags, swipes and zoom



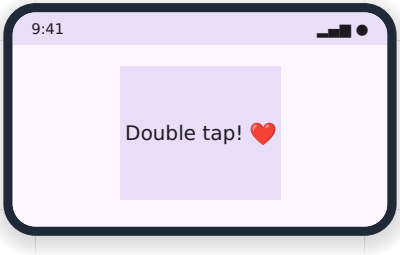
Taps

Taps.kt

```
var text by remember { mutableStateOf("Touch me") }

Box(

Modifier.size(150.dp).background(Color(0xFFE8DEF8))
        .pointerInput(Unit) {
            detectTapGestures(
                onTap = { text = "Tap!" },
                onDoubleTap = { text = "Double tap!
❤️" },
                onLongPress = { text = "Long press
👀" }
            )
        },
    contentAlignment = Alignment.Center
) { Text(text) }
```



📱 What you see

Drag anywhere

Drag.kt

```
@Composable
fun DraggableBall() {
    var offset by remember {
mutableStateOf(Offset.Zero) }

    Box(
        Modifier
            .offset { IntOffset(offset.x.roundToInt(),

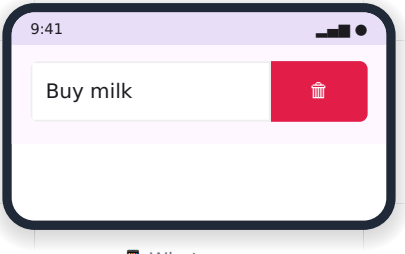
offset.y.roundToInt()) }
            .size(64.dp)
            .clip(CircleShape)
            .background(Color(0xFF4285F4))
            .pointerInput(Unit) {
                detectDragGestures { change,
dragAmount ->
                    change.consume()
                    offset += dragAmount
                }
            }
    )
}
```

Swipe to dismiss

Swipe.kt

```
@Composable
fun SwipeItem(text: String, onDelete: () -> Unit) {
    val state = rememberSwipeToDismissBoxState(
        confirmValueChange = {
            if (it ==
SwipeToDismissBoxValue.EndToStart) {
                onDelete(); true
            } else false
        }
    )
    SwipeToDismissBox(
        state = state,
        enableDismissFromStartToEnd = false,
        backgroundContent = {

Box(Modifier.fillMaxSize().background(Color.Red)
                .padding(16.dp), Alignment.CenterEnd)
{
                Icon(Icons.Default.Delete, "Delete",
                     tint = Color.White)
            }
        }
    ) {
        ListItem(headlineContent = { Text(text) })
    }
}
```



📱 What you see

Pinch to zoom & rotate

Zoom.kt

```
var scale by remember { mutableFloatStateOf(1f) }
var rotation by remember { mutableFloatStateOf(0f) }

Image(
    painterResource(R.drawable.photo), null,
    Modifier
        .graphicsLayer {
            scaleX = scale; scaleY = scale
            rotationZ = rotation
        }
        .pointerInput(Unit) {
            detectTransformGestures { _, _, zoom,
rotate ->
                scale = (scale * zoom).coerceIn(0.5f,
4f)
                rotation += rotate
            }
        }
)
```

> 📌 **Remember**
> Simple gestures have shortcut modifiers: `clickable`, `combinedClickable` (long click), `draggable`, `scrollable` and `toggleable`.

CHAPTER 12

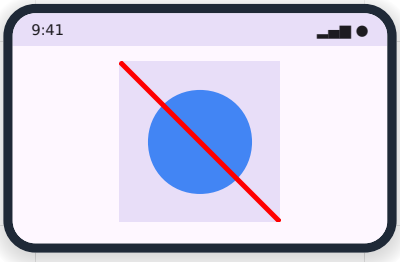
Canvas & Custom Layouts

Draw anything and place children your way

Canvas basics

CanvasBasics.kt

```
Canvas(Modifier.size(200.dp)) {
    // this = DrawScope: size, center, drawXxx()
    drawRect(Color(0xFFE8DEF8))
    drawCircle(Color(0xFF4285F4), radius =
size.minDimension / 3)
    drawLine(
        Color.Red, start = Offset(0f, 0f),
        end = Offset(size.width, size.height),
        strokeWidth = 6f
    )
}
```



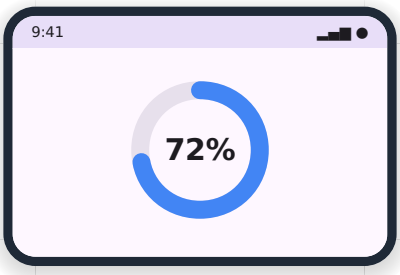
What you see

Function	Draws
drawRect / drawRoundRect	Rectangles
drawCircle / drawOval	Round shapes
drawLine / drawPath	Lines & custom shapes
drawArc	Pie / progress rings
drawText	Text (with TextMeasurer)
drawImage	Bitmaps

Progress ring

ProgressRing.kt

```
@Composable
fun ProgressRing(progress: Float, modifier: Modifier
= Modifier) {
    val animated by animateFloatAsState(progress,
label = "p")
    Box(modifier.size(120.dp), contentAlignment =
Alignment.Center) {
        Canvas(Modifier.fillMaxSize()) {
            val stroke = Stroke(24f, cap =
StrokeCap.Round)
            drawArc(Color(0xFFE8DEF8), 0f, 360f,
                    useCenter = false, style = stroke)
            drawArc(
                brush = Brush.sweepGradient(
                    listOf(Color(0xFF4285F4),
Color(0xFF3DDC84),
                           Color(0xFF4285F4))),
                startAngle = -90f,
                sweepAngle = 360f * animated,
                useCenter = false, style = stroke)
        }
        Text("${(animated * 100).toInt()}%",
             style =
MaterialTheme.typography.headlineSmall)
    }
}
```

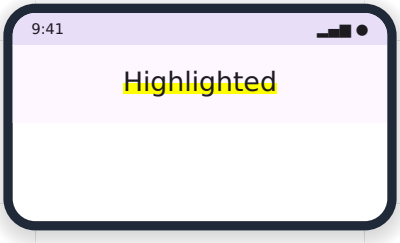


📱 What you see

Draw behind any composable

DrawBehind.kt

```
Text(
    "Highlighted",
    Modifier.drawBehind {
        drawRoundRect(
            Color.Yellow,
            topLeft = Offset(0f, size.height * 0.55f),
            size = Size(size.width, size.height *
0.4f),
            cornerRadius = CornerRadius(8f)
        )
    }
)
```



📱 What you see

Custom Layout

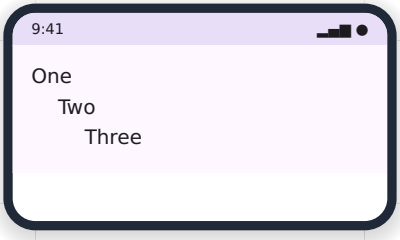
Need a layout that doesn't exist? Measure and place children yourself. Here is a simple **staircase** layout:

Staircase.kt

```kotlin
@Composable
fun Staircase(
    modifier: Modifier = Modifier,
    step: Dp = 20.dp,
    content: @Composable () -> Unit
) {
    Layout(content, modifier) { measurables,
constraints ->
        // 1. measure every child
        val placeables = measurables.map {
it.measure(constraints) }
        val stepPx = step.roundToPx()
        val width = placeables.maxOf { it.width } +
                    stepPx * (placeables.size - 1)
        val height = placeables.sumOf { it.height }

        // 2. decide own size, 3. place children
        layout(width, height) {
            var y = 0
            placeables.forEachIndexed { i, p ->
                p.placeRelative(x = i * stepPx, y = y)
                y += p.height
            }
        }
    }
}

Staircase { Text("One"); Text("Two"); Text("Three") }
```



📱 What you see

📌 **Remember**

Compose measures each child **only once** per layout pass. That rule keeps deep UI trees fast.

CHAPTER 13

Performance & Testing

Keep it smooth and prove it works



Performance checklist

	Rule
✅	Use `key` in lazy lists
✅	Use `remember` for expensive calculations
✅	`derivedStateOf` for rapidly changing state
✅	Read fast-changing state in lambdas (`offset { }`, `graphicsLayer { }`)
✅	Prefer immutable data (`val`, `List`) in UI state
✅	Test performance in **release** builds, not debug
❌	Never sort / filter big lists inside a composable without `remember`
❌	No I/O or heavy work in composition

Deferring state reads

DeferRead.kt

```
// ❌ recomposes the whole Box on every scroll pixel
Box(Modifier.offset(y = scroll.value.dp))

// ✅ only re-places; no recomposition
Box(Modifier.offset { IntOffset(0, scroll.value) })
```

Stability

Stability.kt

```kotlin
// Compose can skip this when 'user' hasn't changed,
// because all its properties are stable vals
@Immutable
data class User(val name: String, val tags:
List<String>)

@Composable
fun UserCard(user: User) { /* ... */ }
```

> 💡 **Tip**
> Use **Layout Inspector** in Android Studio to see recomposition counts for each composable — numbers that keep climbing reveal wasted work.

UI tests

androidTest/CounterTest.kt

```
class CounterTest {
    @get:Rule
    val rule = createComposeRule()

    @Test
    fun clickingPlus_increasesCount() {
        rule.setContent { Counter() }

        rule.onNodeWithText("0").assertIsDisplayed()
        rule.onNodeWithText("+").performClick()
        rule.onNodeWithText("1").assertIsDisplayed()
    }

    @Test
    fun loginButton_disabledUntilValid() {
        rule.setContent { LoginForm(onLogin = { _, _ -
> }) }

        rule.onNodeWithText("Log
in").assertIsNotEnabled()

rule.onNodeWithText("Email").performTextInput("a@b.com")

rule.onNodeWithText("Password").performTextInput("secret
        rule.onNodeWithText("Log
in").assertIsEnabled()
    }
}
```

```
▶ Test results
✔ clickingPlus_increasesCount
✔ loginButton_disabledUntilValid
2 tests passed
```

Finding nodes

API	Use
onNodeWithText("Hi")	By visible text
onNodeWithContentDescription("Like")	By accessibility label
onNodeWithTag("list")	By Modifier.testTag("list")
performClick() / performScrollTo()	Actions
assertExists() / assertIsDisplayed()	Checks

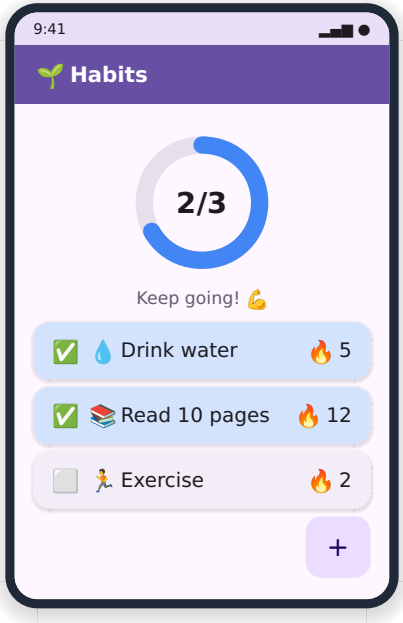
> 📌 **Remember**
> Good accessibility (`contentDescription` , real text labels) makes your app easier to test **and** easier to use for everyone.

CHAPTER 14

Mini Project: Habit Tracker

Animations, Canvas, lists and state together

A single-file app combining a **Canvas progress ring**, an **animated counter**, **animated list items**, a **dialog** and a **state holder**.



📱 What you see

MainActivity.kt

```
data class Habit(val id: Int, val name: String,
                 val streak: Int = 0, val doneToday:
Boolean = false)

class HabitState {
    private var nextId = 4
```

```
    val habits = mutableStateListOf(
        Habit(1, "💧 Drink water", 4),
        Habit(2, "📚 Read 10 pages", 11),
        Habit(3, "🏃 Exercise", 2))

    val progress: Float
        get() = if (habits.isEmpty()) 0f
                else habits.count { it.doneToday } /
habits.size.toFloat()

    fun toggle(h: Habit) {
        val i = habits.indexOf(h)
        habits[i] = h.copy(
            doneToday = !h.doneToday,
            streak = h.streak + if (h.doneToday) -1
else 1)
    }
    fun add(name: String) { habits += Habit(nextId++,
name) }
    fun remove(h: Habit) { habits -= h }
}

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState:
Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent { MaterialTheme { HabitApp() } }
    }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun HabitApp() {
    val state = remember { HabitState() }
    var showAdd by remember { mutableStateOf(false) }

    Scaffold(
        topBar = { TopAppBar(title = { Text("🌱
Habits") }) },
        floatingActionButton = {
            FloatingActionButton(onClick = { showAdd
= true }) {
                Icon(Icons.Default.Add, "Add habit")
            }
        }
```

```
    ) { padding ->
        LazyColumn(
            Modifier.padding(padding),
            contentPadding = PaddingValues(16.dp),
            verticalArrangement =
Arrangement.spacedBy(8.dp)
        ) {
            item { Header(state) }
            items(state.habits, key = { it.id }) {
habit ->
                HabitRow(habit,
                    onToggle = {
state.toggle(habit) },
                    onDelete = {
state.remove(habit) },
                    modifier = Modifier.animateItem())
            }
        }
    }
    if (showAdd) AddHabitDialog(
        onAdd = { state.add(it); showAdd = false },
        onDismiss = { showAdd = false })
}

@Composable
fun Header(state: HabitState) {
    val done = state.habits.count { it.doneToday }
    Column(Modifier.fillMaxWidth(),
           horizontalAlignment =
Alignment.CenterHorizontally) {
        Box(contentAlignment = Alignment.Center) {
            ProgressRing(state.progress)   // from
Chapter 12
        }
        AnimatedContent(done, label = "done") {
            Text("$it of ${state.habits.size} done
today",
                 style =
MaterialTheme.typography.titleMedium)
        }
        Text(if (state.progress == 1f) "Perfect day!
🎉"
             else "Keep going! 💪")
        Spacer(Modifier.height(8.dp))
    }
}
```

```
@Composable
fun HabitRow(habit: Habit, onToggle: () -> Unit,
             onDelete: () -> Unit, modifier: Modifier
= Modifier) {
    val bg by animateColorAsState(
        if (habit.doneToday)
            MaterialTheme.colorScheme.primaryContainer
        else MaterialTheme.colorScheme.surfaceVariant,
        label = "bg")
    Card(
        modifier.fillMaxWidth(),
        colors =
CardDefaults.cardColors(containerColor = bg)
    ) {
        Row(Modifier.padding(8.dp),
            verticalAlignment =
Alignment.CenterVertically) {
            Checkbox(habit.doneToday, { onToggle() })
            Text(habit.name, Modifier.weight(1f))
            Text("🔥 ${habit.streak}")
            IconButton(onClick = onDelete) {
                Icon(Icons.Default.Close, "Delete")
            }
        }
    }
}

@Composable
fun AddHabitDialog(onAdd: (String) -> Unit,
onDismiss: () -> Unit) {
    var name by remember { mutableStateOf("") }
    AlertDialog(
        onDismissRequest = onDismiss,
        title = { Text("New habit") },
        text = {
            OutlinedTextField(name, { name = it },
                placeholder = { Text("e.g. 🧘
Meditate") },
                singleLine = true)
        },
        confirmButton = {
            TextButton(onClick = {
onAdd(name.trim()) },
                       enabled = name.isNotBlank()) {
Text("Add") }
```

```
        },
        dismissButton = {
            TextButton(onClick = onDismiss) {
Text("Cancel") }
        }
    )
}
```

✍️ Try it yourself

1) Make the whole card **swipe to delete** (Chapter 11).
2) Shake the ring with an animation when the day is complete.
3) Move `HabitState` into a ViewModel and save habits with Room.

⚡ Compose Cheat Sheet

The APIs you'll use every day

Task	Code
State	`var x by remember { mutableStateOf(0) }`
Saved state	`rememberSaveable { }`
Derived	`remember { derivedStateOf { } }`
Layout	`Column` · `Row` · `Box` · `FlowRow`
Share space	`Modifier.weight(1f)`
Lists	`LazyColumn { items(list, key = { it.id }) { } }`
Grid	`LazyVerticalGrid(GridCells.Adaptive(100.dp))`
Pager	`HorizontalPager(rememberPagerState { n })`
Shape	`Modifier.clip(RoundedCornerShape(12.dp))`
Theme	`MaterialTheme.colorScheme / typography`
Effect	`LaunchedEffect(key) { }`
Cleanup	`DisposableEffect(key) { onDispose { } }`
Scope	`rememberCoroutineScope()`
Animate value	`animateColorAsState(target)`
Show/hide	`AnimatedVisibility(visible) { }`
Swap	`AnimatedContent(state) { }`
Gestures	`pointerInput(Unit) { detectTapGestures() }`
Draw	`Canvas(mod) { drawCircle() }`
Dialog	`AlertDialog(onDismissRequest, confirmButton)`
Test	`rule.onNodeWithText("+").performClick()`

ANSWERS

Exercise Solutions



For the mini project challenges

1) Swipe to delete

SwipeHabit.kt

```
items(state.habits, key = { it.id }) { habit ->
    val dismiss = rememberSwipeToDismissBoxState(
        confirmValueChange = {
            if (it != SwipeToDismissBoxValue.Settled)
{
                state.remove(habit); true
            } else false
        })
    SwipeToDismissBox(
        state = dismiss,
        modifier = Modifier.animateItem(),
        backgroundContent = {
            Box(Modifier.fillMaxSize()
                .background(Color(0xFFE24462),

RoundedCornerShape(12.dp)))
        }
    ) {
        HabitRow(habit, { state.toggle(habit) },
                 { state.remove(habit) })
    }
}
```

2) Celebrate a perfect day

Celebrate.kt

```
val rotation = remember { Animatable(0f) }

LaunchedEffect(state.progress == 1f) {
    if (state.progress == 1f) {
        repeat(3) {
            rotation.animateTo(10f, tween(60))
            rotation.animateTo(-10f, tween(60))
        }
        rotation.animateTo(0f, spring())
    }
}

ProgressRing(
    state.progress,
    Modifier.graphicsLayer { rotationZ =
rotation.value }
)
```

3) Move to a ViewModel

HabitViewModel.kt

```
class HabitViewModel(private val dao: HabitDao) :
ViewModel() {
    val habits = dao.getAll().stateIn(
        viewModelScope,
SharingStarted.WhileSubscribed(5_000),
        emptyList())

    fun toggle(h: Habit) = viewModelScope.launch {
        dao.update(h.copy(doneToday = !h.doneToday,
            streak = h.streak + if (h.doneToday) -1
else 1))
    }
}

@Composable
fun HabitApp(vm: HabitViewModel = hiltViewModel()) {
    val habits by
vm.habits.collectAsStateWithLifecycle()
    // ... same UI, but calls vm.toggle(habit)
}
```

🎉 You've mastered Jetpack Compose!
Next steps: explore Compose Multiplatform (iOS + desktop with the same code), shared element transitions, Glance for widgets, and the official **developer.android.com/compose** samples.